



It's Your Life

with





PREVIOUSLY
ON...



KB It's your life

🌐 Coding with Fun

🌐 Coding with us



© 2025 by Tetz in KB6





JS





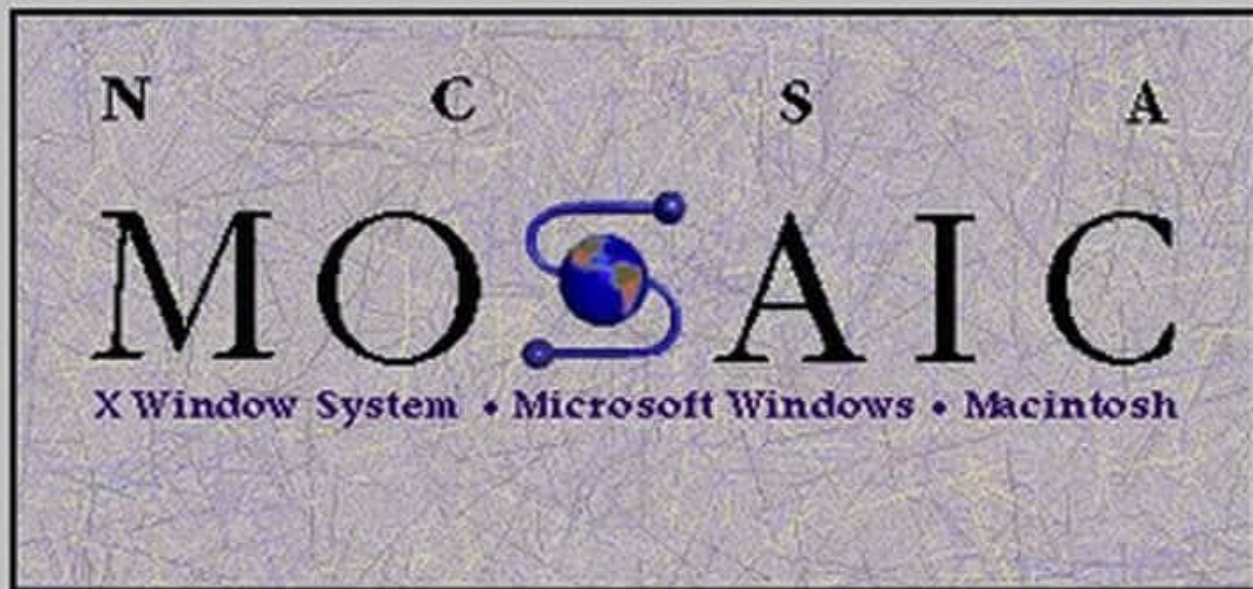
File Edit Options Navigate Hotlist Annotate



Document Title: | NCSA Mosaic Home Page|

Document URL: <http://www.ncsa.uiuc.edu/SDG/Software/Mosaic/NCSAMosaicHome.html>

1993년



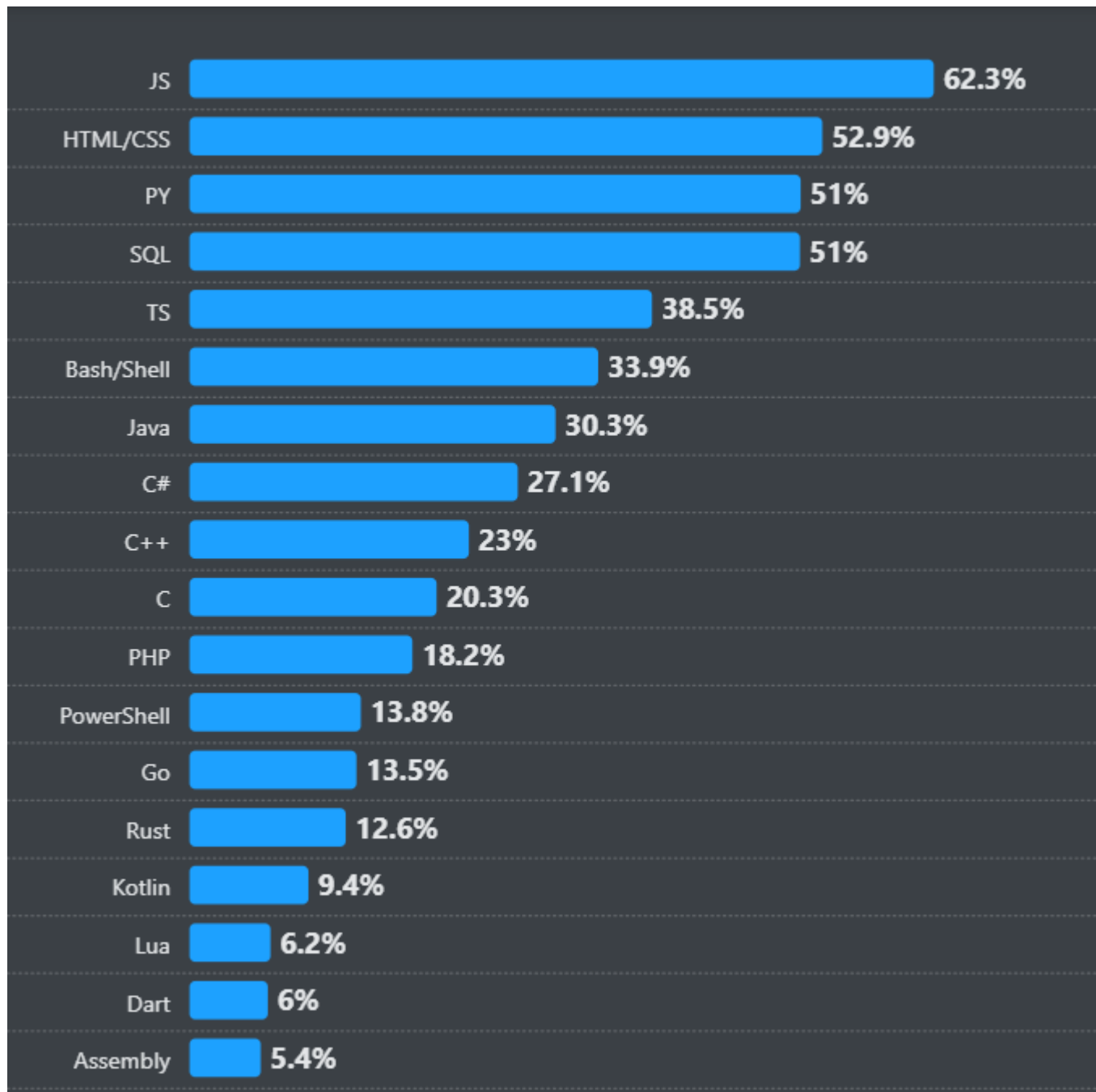
Welcome to NCSA Mosaic, an Internet information browser and [World Wide Web](#)
Mosaic was developed at the [National Center for Supercomputing Application](#)
[University of Illinois](#) in --> Urbana-Champaign. NCSA Mosaic software is co
The Board of Trustees of the University of Illinois (UI), and ownership re



브랜던 아이크가 1995년에
다양한 브라우저에서
동작하기 위해

자바를 참고하여
스크립트 타입의 언어를 개발!







pay

N

메일

카페

블로그

스토어

뉴스

증권

부동산

지도

웹툰

치지직

정관장

새로운 시작, 건강하게
AD 정관장 불맛이 ~30%

AD

솔직히 퇴근하고 딱 1장만 합시다.

영어 손 뗀지 오래된 분들 대모집 **단 11시까지만 500원 >**

본 혜택은 이후에도 동일하게 적용될 수 있음 / 연말 시 최초 1회

뉴스스탠드

연론사편집

엔터

스포츠

경제

쇼핑투데이

전체언론사

연합뉴스

국민의힘 42.7%

민주 41.0%

정권 교체 50.4%

정권 연장 4...

뉴스스탠드

뉴스홈

MTQ 마니투데이	서울신문	MBC	KBS WORLD	뉴데일리	한겨레
BLOTER	아이뉴스24	미디어오늘	Chosun	일간스포츠	스포츠서울
경향신문	노컷뉴스	데일리안	Insight	매경웹스	VERITAS
hankyung JOB&JOY	뉴스방관	TVREPORT	2025 오늘	이코노미뉴스	대한항공호텔

<

언론사 더보기 1/4

>

쇼핑

맨즈

원플러스

쇼핑라이브

1/26 < >

네이버를 더 안전하고 편리하게 이용하세요

NAVER 로그인

아이디 찾기 | 비밀번호 찾기 | 회원가입

한국직업능력진흥원 무료교육

한국직업능력진흥원
2025 최신 인기자격증
무료수강

한국직업능력진흥원 >

독박 속았수다.

아이유 & 박보검
넷플릭스 인생작 왔다!
내미여 행배심員 4,900만 시청 >

많이 찾는
아이템
추천해요

10043565&scale=5&size=Opp&size=507M&Ka56



JS 기초!



표기법

dash-case(kebab-case)

snake_case

camelCase

ParcelCase



The-qiuck-borwn-fox-jupms-oevr-the-
lzay-dog

옳은예 : 서울시 체육회
나쁜예 : 서울 시체 육회

옳은예 : 서울시 장애인 복지관
나쁜예 : 서울시장 애인 복지관

옳은예 : 무지개 같은 사장님
나쁜예 : 무지 개같은 사장님



~~Var(iable)~~



외알도?





```
// var  
var name = "tetz";  
var name = "LHS";  
  
console.log(name);
```

```
// var  
var tetz = "LHS";  
  
if (tetz == "LHS") {  
    var result = true;  
} else {  
    var result = false;  
}  
  
console.log(result);
```

<https://www.youtube.com/shorts/mgdCHjJjR4M?feature=share>



(예전 JS 개발현장)



데이터 종류(자료형)

String

Number

Boolean

Undefined

Null

Object

Array

자료형을 알려주는 **typeof**



- 해당 자료형이 어떤 것인지 알려주는 착한 친구입니다!

```
let num = 1;
let str = "이호석";
let bool = true;
let undef = undefined;
let nul = null;
let arr = [1, 2, 3];
let obj = {
  num: 1,
  str: "String"
}

console.log(typeof num);
console.log(typeof str);
console.log(typeof bool);
console.log(typeof undef);
console.log(typeof nul);
console.log(typeof arr);
console.log(typeof obj);
```

number

string

boolean

undefined

object

object

object

문자와 변수를 혼용해서 쓰고 싶을 때!



```
for(let i = 0; i < nameArr.length; i++) {  
    console.log(` ${i+1} 번째 이름은 ${nameArr[i]} 입니다` );  
}
```

1 번째 이름은 안은현 입니다

2 번째 이름은 강병현 입니다

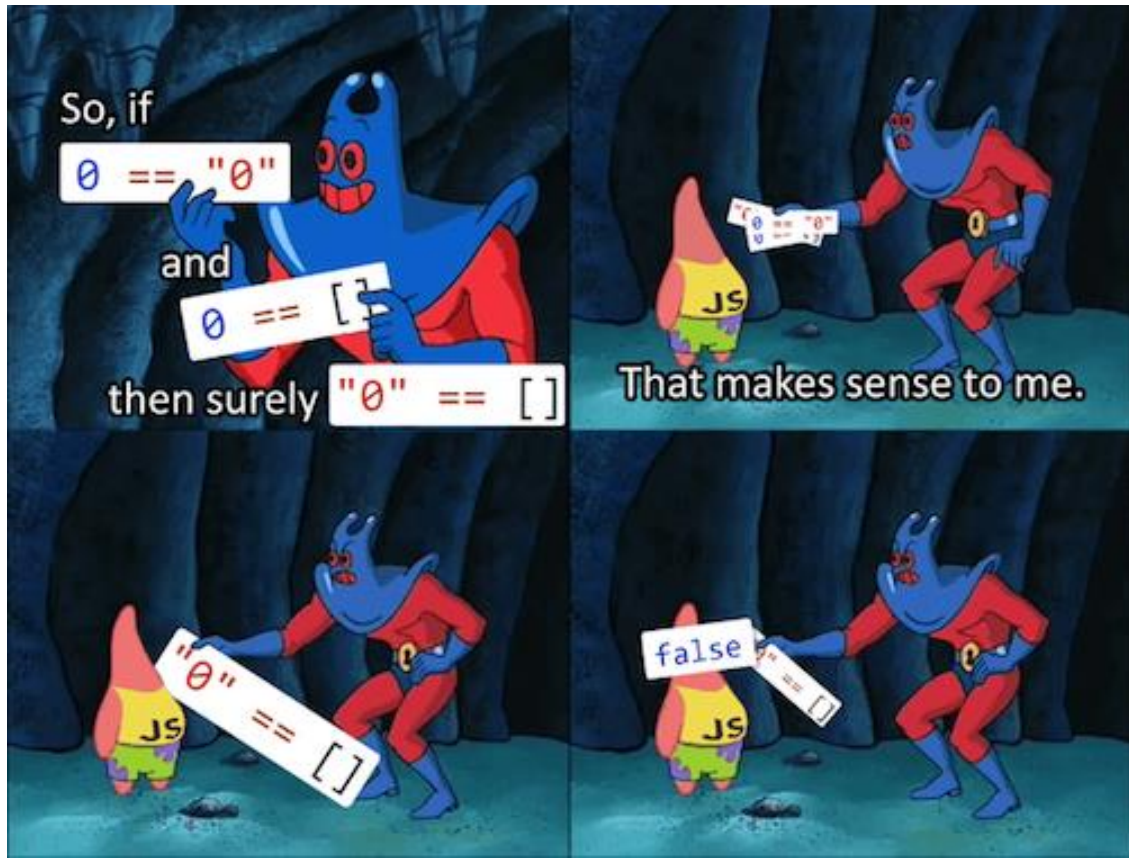
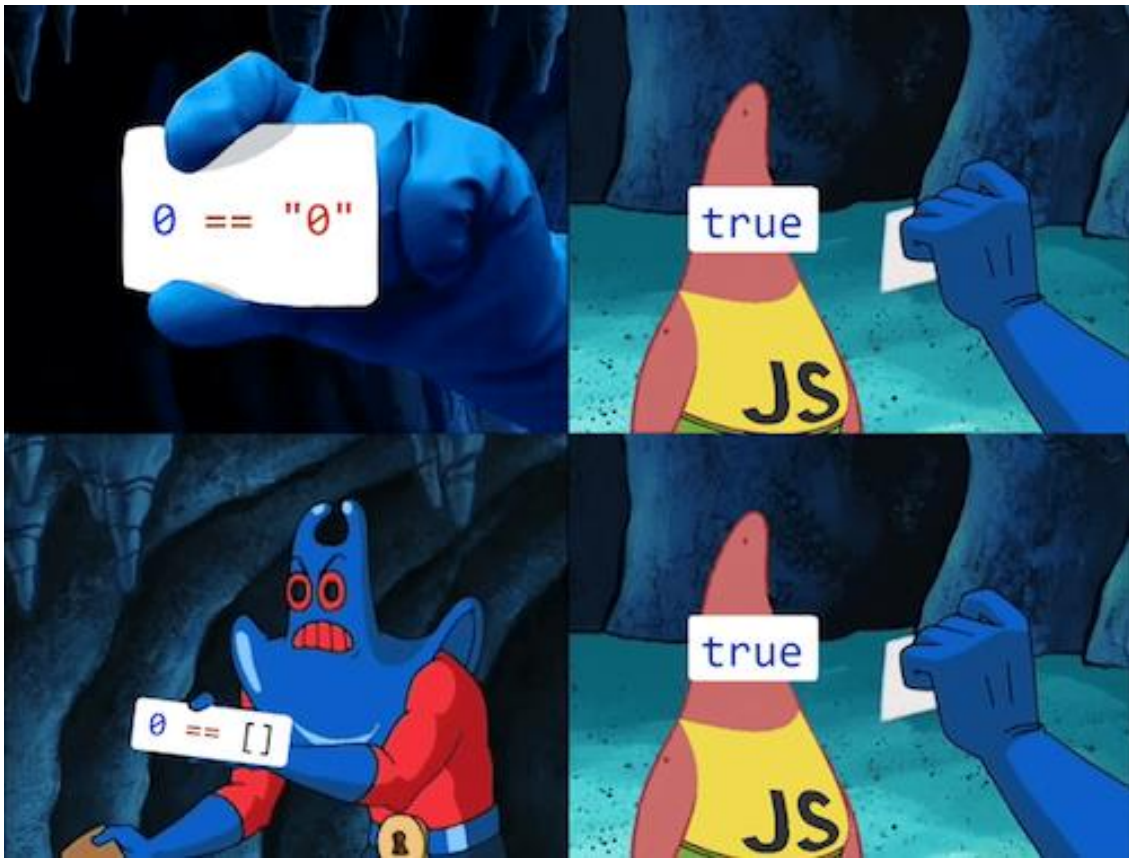
3 번째 이름은 염다빈 입니다

4 번째 이름은 권다연 입니다

5 번째 이름은 이혜원 입니다

6 번째 이름은 김은지 입니다

7 번째 이름은 이준범 입니다



⋮

Console

What's New

⊘

top

▼

Filter

> 2 + 2

< 4

> "2" + "2"

< "22"

> 2 + 2 - 2

< 2

> "2" + "2" - "2"

< 20



> typeof NaN

< "number"

> 9999999999999999

< 10000000000000000

> 0.5+0.1==0.6

< true

> 0.1+0.2==0.3

< false

> Math.max()

< -Infinity

> Math.min()

< Infinity

> []+[]

< ""

> []+{}

< "[object Object]"

> {}+[]

< 0

> true+true+true===3

< true

> true-true

< 0

> true==1

< true

> true===1

< false

> (!+[]+[]+![]).length

< 9

> 9+"1"

< "91"

> 91-"1"

< 90

> []==0

< true



Thanks for inventing Javascript



동등 연산자

비교하기 전에
공통 타입으로 변환 후
'값'만 비교



일차 연산자

타입 변환 없이 비교
값과 타입이 모두 같아야
true 를 리턴





성적을 구하는 프로그램 만들기

```
let mathScore = prompt("수학 점수를 입력 하세요");  
let engScore = prompt("영어 점수를 입력 하세요");  
  
let avg = (mathScore + engScore) / 2;  
  
console.log(avg);
```

- 그런데 결과값이 좀 이상하죠?
- **Prompt** 로 입력 받은 값은 **"문자"**로 저장이 됩니다!
- $"80" + "50" = "8050" \rightarrow "8050" / 2 \rightarrow 4025$

SC

박권라



시자아아아악-
하졌습니다!!



여러분 사실 코딩은
X 와 Y 의 반복 입니다!

X, Y 는 무엇일까요?

그런데 말입니다



조건문

IF / ELSE



```
if ( 조건1 ) {  
    // 조건1이 참이라면 실행  
} else {  
    // 조건1이 거짓이라면 실행  
}
```

IF / ELSE IF / ELSE



```
if ( 조건1 ) {  
    // 조건1이 참이라면 실행  
} else if ( 조건2 ) {  
    // 조건2가 참이라면 실행  
} else {  
    // 조건 1과 2가 모두 참이 아닐 때 실행  
}
```

IF 중첩



```
if ( 조건1 ) {  
    if ( 조건2 ) {  
        //실행  
    } else {  
        //실행2  
    }  
}
```



```
let isShow = true;
let checked = false;

if (isShow) {
  console.log('Show!'); // Show!
}

if (checked) {
  console.log('Checked!');
}
```



```
let isShow = true;

if (isShow) {
    console.log('Show!');
} else {
    console.log('Hide?');
}
```




조건문

Switch



```
switch ( 변수 ) {  
    case 값1:  
        // 변수와 값1이 일치하면 실행  
        break;  
    case 값2:  
        // 변수와 값2가 일치하면 실행  
        break;  
    default:  
        //일치하는 값이 없을 때 실행  
        break;  
}
```



```
// Switch
let day;
switch (new Date().getDay()) {
  case 0:
    day = "일요일";
    break;
  case 1:
    day = "월요일";
    break;
  case 2:
    day = "화요일";
    break;
  case 3:
    day = "수요일";
    break;
  case 4:
    day = "목요일";
    break;
  case 5:
    day = "금요일";
    break;
  case 6:
    day = "토요일";
    break;
}

console.log(day);
```

A screenshot of a terminal window with a dark background. The text '월요일' (Monday) is displayed in a white, monospaced font. Below the text, there is a blue prompt character and a vertical cursor line.

월요일



3항 연산자



IF 문을 간단하게 표현하는 방법

- 조건식 ? 조건이 참인 경우 실행 : 조건이 거짓인 경우 실행;
- 한 줄로 간단히 표현 가능!

```
// 3항 연산자
let tetzName = "LHS";

if (tetzName == "LHS") {
  console.log("맞았어요!!");
} else {
  console.log("틀렸어요 bb");
}

tetzName != "LHS" ? Console.log("맞았어요!!") : console.log("틀렸어요 ππ");
```



하지만 드라군이
출동하면 어떨까?

드!

라!

군!



```
return man && man.childrens
  ? (man.childrens[0].name === 'Johnny')
    ? (man.childrens[0].age > 19
      ? 'johnny old more than 19'
      : (man.childrens[0].age === 14
        ? 'johnny old more than 14, less than 19'
        : 'johnny is baby'
      )
    )
  : (man.childrens[0].name === 'Monica'
    ? 'Monica'
    : 'no Monica'
  )
: 'No children';
```



```
type UnpackMenuNames<T extends ReadonlyArray<MenuItem>> =  
  T extends ReadonlyArray<infer U>  
    ? U extends MainMenu  
      ? U['subMenus'] extends infer V  
        ? V extends  
          ? Unpack  
            : U['name']  
          : never  
        : U extends  
          ? U['name']  
          : never  
        : never;  
: never;
```



실습, 로그인 프로그램 작성하기



- 다음장의 코드를 이용해서 로그인을 처리하는 프로그램을 완성해 주세요
- 조건을 처리하는 방법은 원하시는 방법을 자유롭게 사용하셔도 됩니다
- ID, PW 는 연속으로 입력을 받습니다
- ID, PW 가 전부 맞으면 console.log 를 통해 “로그인 성공” 메시지 출력
- ID 가 틀리면 “ID 가 틀렸습니다” 로그 출력
- PW 가 틀리면 “PW 가 틀렸습니다” 로그 출력
- ID, PW 가 전부 틀리면 “망했어요” 로그 출력

```
<script>
  const id = 'tetz';
  const pw = '1324';

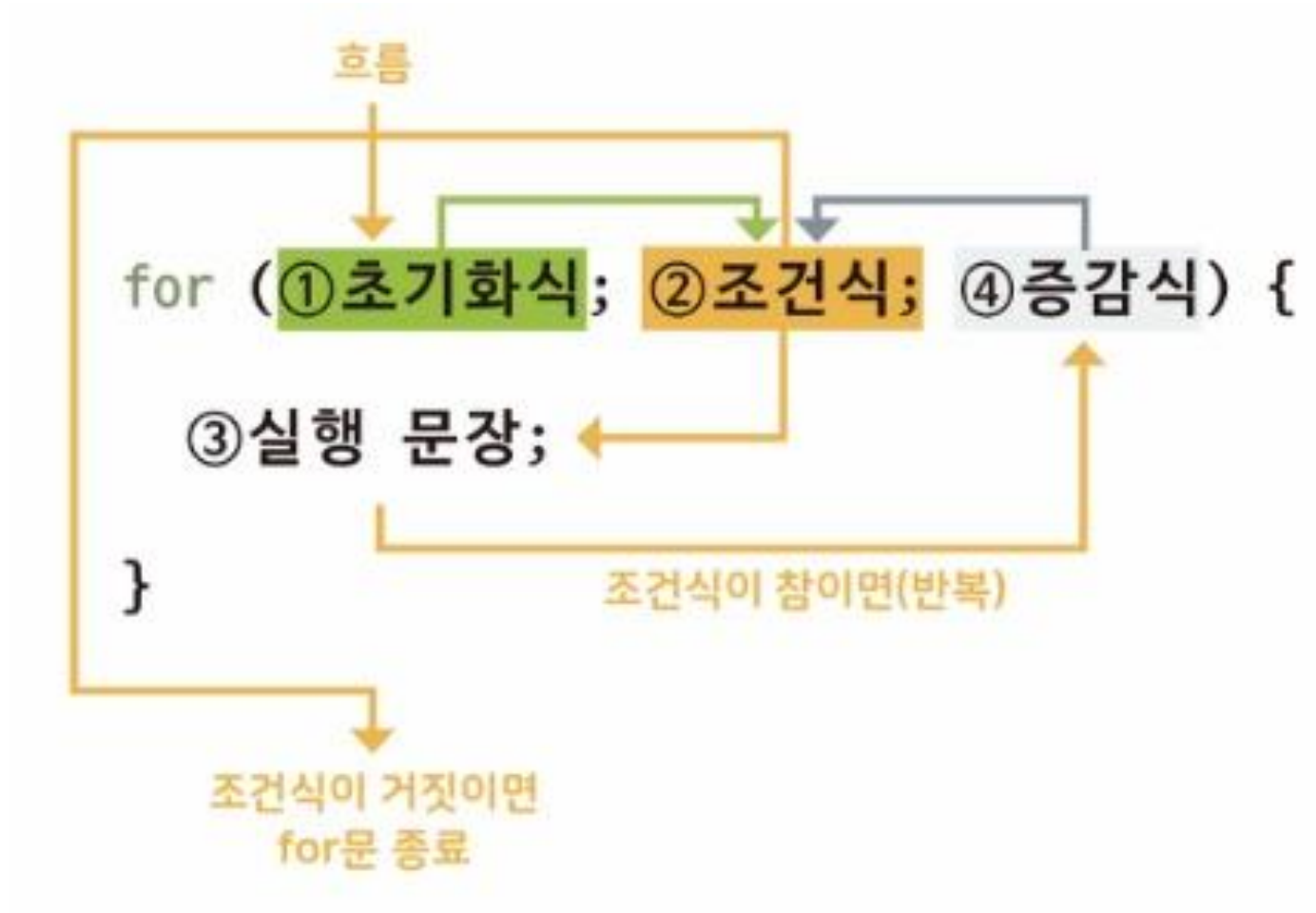
  const idInput = prompt('ID 를 입력 하세요');
  const pwInput = prompt('비밀번호를 입력하세요');
</script>
```





반복문

For, while



For 문



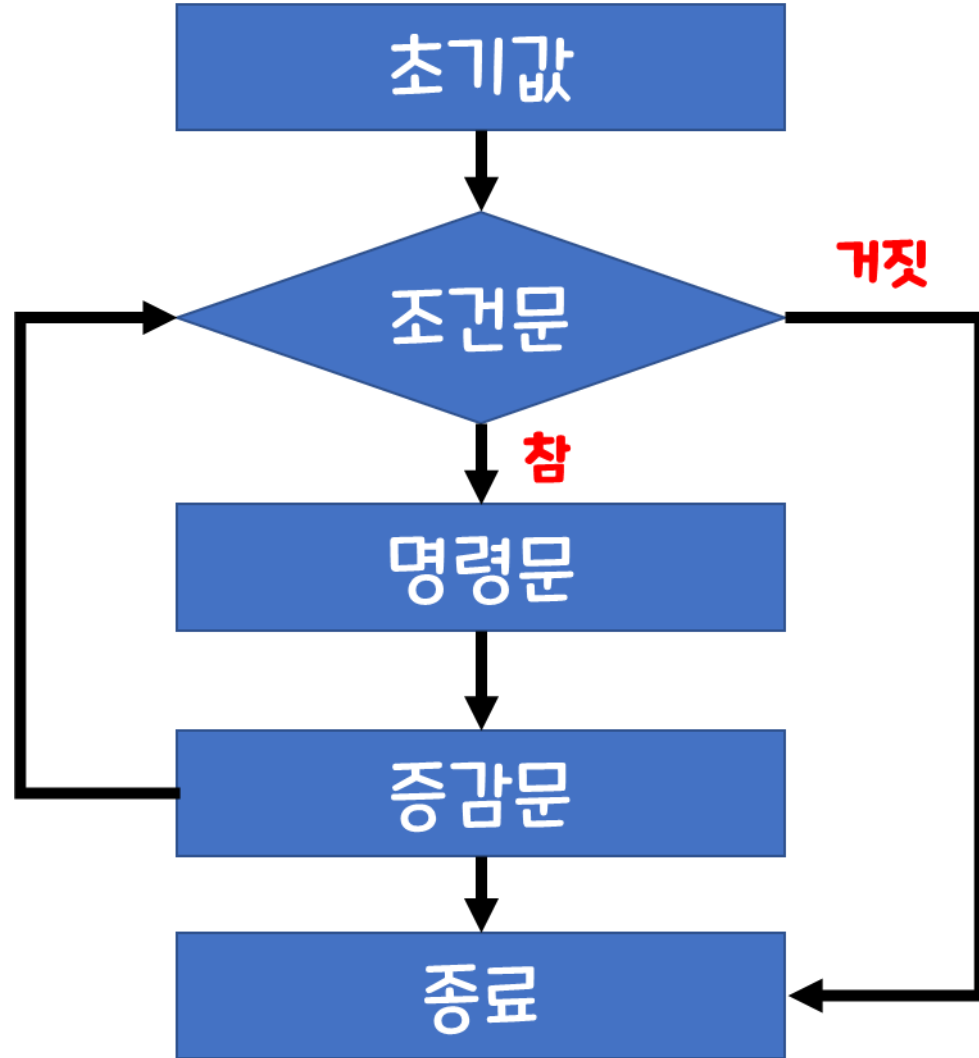
- for(초기 상태 설정; 조건문; 증감문) {

실행할 코드

}

```
// for 문
for(let index = 0; index < 10; index++) {
  console.log("인사를 ", index+1, "번째 드립니다! 😊");
}
```

인사를	1	번째	드립니다!	😊
인사를	2	번째	드립니다!	😊
인사를	3	번째	드립니다!	😊
인사를	4	번째	드립니다!	😊
인사를	5	번째	드립니다!	😊
인사를	6	번째	드립니다!	😊
인사를	7	번째	드립니다!	😊
인사를	8	번째	드립니다!	😊
인사를	9	번째	드립니다!	😊
인사를	10	번째	드립니다!	😊



2중 For 문



- For 문을 2번 중첩해서 사용하는 반복문!

```
for (let i = 0; i < 3; i++) {  
  console.log(`상위 for 문입니다!`, ${i + 1} 번째 실행중`);  
  for (let j = 0; j < 5; j++) {  
    console.log(`하위 for 문입니다!`, ${j + 1} 번째 실행중`);  
  }  
}
```

```
상위 for 문입니다!, 0 번째 실행중  
하위 for 문입니다!, 0 번째 실행중  
하위 for 문입니다!, 1 번째 실행중  
하위 for 문입니다!, 2 번째 실행중  
하위 for 문입니다!, 3 번째 실행중  
하위 for 문입니다!, 4 번째 실행중  
상위 for 문입니다!, 1 번째 실행중  
하위 for 문입니다!, 0 번째 실행중  
하위 for 문입니다!, 1 번째 실행중  
하위 for 문입니다!, 2 번째 실행중  
하위 for 문입니다!, 3 번째 실행중  
하위 for 문입니다!, 2 번째 실행중  
하위 for 문입니다!, 3 번째 실행중  
하위 for 문입니다!, 4 번째 실행중  
하위 for 문입니다!, 5 번째 실행중  
상위 for 문입니다!, 3 번째 실행중  
하위 for 문입니다!, 1 번째 실행중  
하위 for 문입니다!, 2 번째 실행중  
하위 for 문입니다!, 3 번째 실행중  
하위 for 문입니다!, 4 번째 실행중  
하위 for 문입니다!, 5 번째 실행중
```



반복문

while



while

```
let i = 0;
```

```
while (i < 10) {
```

```
// 코드
```

```
}
```



while

```
let i = 0;
```

```
while (i < 10) {  
    // 코드  
    i++;  
}
```

while 문



- while(조건문) {

실행할 코드(명령문)

}

- For 문과는 달리 조건을 변경하는 구문이 기본적으로 포함이 되어 있지 않기 때문에 무한 루프의 가능성이 농후!
- 주의하여 사용 필요



```
// while 문

// 1번 타입, 조건문을 사용
let index = 0;

while (index < 10) {
  console.log("인사를 ", index + 1, "번째 드립니다! 😊");
  index++;
}

// 2번 타입, 조건문을 사용하지 않고 if 문 + break 사용
let index2 = 0;

while (true) {
  console.log("절을 ", index2 + 1, "번째 드립니다! 😊");
  index2++;
  if (index2 == 10) {
    break;
  }
}
```

인사를	1	번째	드립니다!	😊
인사를	2	번째	드립니다!	😊
인사를	3	번째	드립니다!	😊
인사를	4	번째	드립니다!	😊
인사를	5	번째	드립니다!	😊
인사를	6	번째	드립니다!	😊
인사를	7	번째	드립니다!	😊
인사를	8	번째	드립니다!	😊
인사를	9	번째	드립니다!	😊
인사를	10	번째	드립니다!	😊
절을	1	번째	드립니다!	😊
절을	2	번째	드립니다!	😊
절을	3	번째	드립니다!	😊
절을	4	번째	드립니다!	😊
절을	5	번째	드립니다!	😊
절을	6	번째	드립니다!	😊
절을	7	번째	드립니다!	😊
절을	8	번째	드립니다!	😊
절을	9	번째	드립니다!	😊
절을	10	번째	드립니다!	😊



반복문

do - while



do.. while

```
let i = 0;
```

```
do {  
    // 코드  
    i++;  
} while (i < 10)
```



do.. while

```
let i = 0;
```

```
do {
```

```
    // 코드
```

```
    i++;
```

```
} while (i < 10)
```

코드가

최소

1번 실행



반복문

제어



break, continue

break

: 멈추고 빠져나옴

continue

: 멈추고 다음 반복으로 진행

break



- 반복문을 멈추고 밖으로 빠져 나감

```
// break

for(let i = 0; i < 100; i++) {
  if(i==10) {
    console.log("멈춰!");
    break;
  }
  console.log(i);
}
```

0
1
2
3
4
5
6
7
8
9
멈춰!

continue



- 반복문을 한 번만 멈추고 다음으로 진행

```
// continue
for (let i = 0; i < 10; i++) {
  if (i % 2 == 0) continue;
  console.log(`${i} 번 입니다!`);
}
```

1 번 입니다!

3 번 입니다!

5 번 입니다!

7 번 입니다!

9 번 입니다!

> |

실습, 구구단 출력하기



- 1단부터 9단까지의 구구단을 출력하는 코드를 완성해 주세요
- 단, 4단은 생략해 주세요
- For, while, do-while 등등 원하시는 반복문을 사용하시면 됩니다



함수

Function!



함수

특정 동작(기능)을 수행하는 일부 코드의 집합(부분)
function



함수(function)

함수 함수명 매개변수

```
function sayHello(name){  
    console.log(`Hello, ${name}`);  
}  
  
sayHello('Mike');
```



```
// 함수 선언  
function helloFunc() {  
    // 실행 코드  
    console.log(1234);  
}  
  
// 함수 호출  
helloFunc(); // 1234
```



```
function returnFunc() {  
    return 123;  
}
```

```
let a = returnFunc();
```

```
console.log(a); // 123
```



// 함수 선언!

```
function sum(a, b) { // a와 b는 매개변수(Parameters)
    return a + b;
}
```

// 재사용!

```
let a = sum(1, 2); // 1과 2는 인수(Arguments)
let b = sum(7, 12);
let c = sum(2, 4);

console.log(a, b, c); // 3, 19, 6
```



```
// 기명(이름이 있는) 함수  
// 함수 선언!  
function hello() {  
    console.log('Hello~');  
}
```

```
// 익명(이름이 없는) 함수  
// 함수 표현!  
let world = function () {  
    console.log('World~');  
}
```

```
// 함수 호출!  
hello(); // Hello~  
world(); // World~
```




매개변수와 인자



```
function sum(num1, num2) {  
  return num1 + num2;  
}
```

```
sum(1, 2);
```

매개 변수

인자

매개변수와 인자



- 인자 : 함수를 호출 시에 실제로 전달 되는 값
- 매개 변수 : 함수를 정의할 때, 값을 전달하기 위해 사용하는 변수 → 인자를 받아주는 변수

매개 변수가 없는 함수!



- 그냥 실행만 됩니다!

```
// 매개 변수가 없는 함수
function showErr() {
    console.log("에러가 발생 했습니다!");
}

showErr();
```

매개 변수가 있는 함수!



- 함수의 출력 내용을 바꾸고 싶다면? 매번 다른 함수를 선언 해서 사용해야 할까요?
- 매개 변수를 전달하여 함수의 실행 값을 변경 할 수 있습니다!

```
function sayHello(name) {  
    console.log(`Hello, ${name}`);  
}  
  
sayHello("Tetz");  
sayHello("Pororo");  
sayHello("Son");
```

```
Hello, Tetz  
Hello, Pororo  
Hello, Son
```

매개 변수, default



- 매개 변수가 있는 함수인데 매개 변수 전달을 안한다면?

```
function sayHello(name) {  
    console.log(`Hello, ${name}`);  
}  
  
sayHello();
```

```
Hello, undefined
```

- 기본 값 설정이 가능합니다!

```
function sayHello(name = "friend") {  
    console.log(`Hello, ${name}`);  
}  
  
sayHello();
```

```
Hello, friend
```



함수!

선언 방법들



함수 선언문 vs 함수 표현식

```
function sayHello(){  
    console.log('Hello');  
}
```

함수 선언문



함수 선언문 vs 함수 표현식

```
let sayHello = function(){  
  console.log('Hello');  
}
```

함수 표현식



함수 선언문 : 어디서든 호출 가능

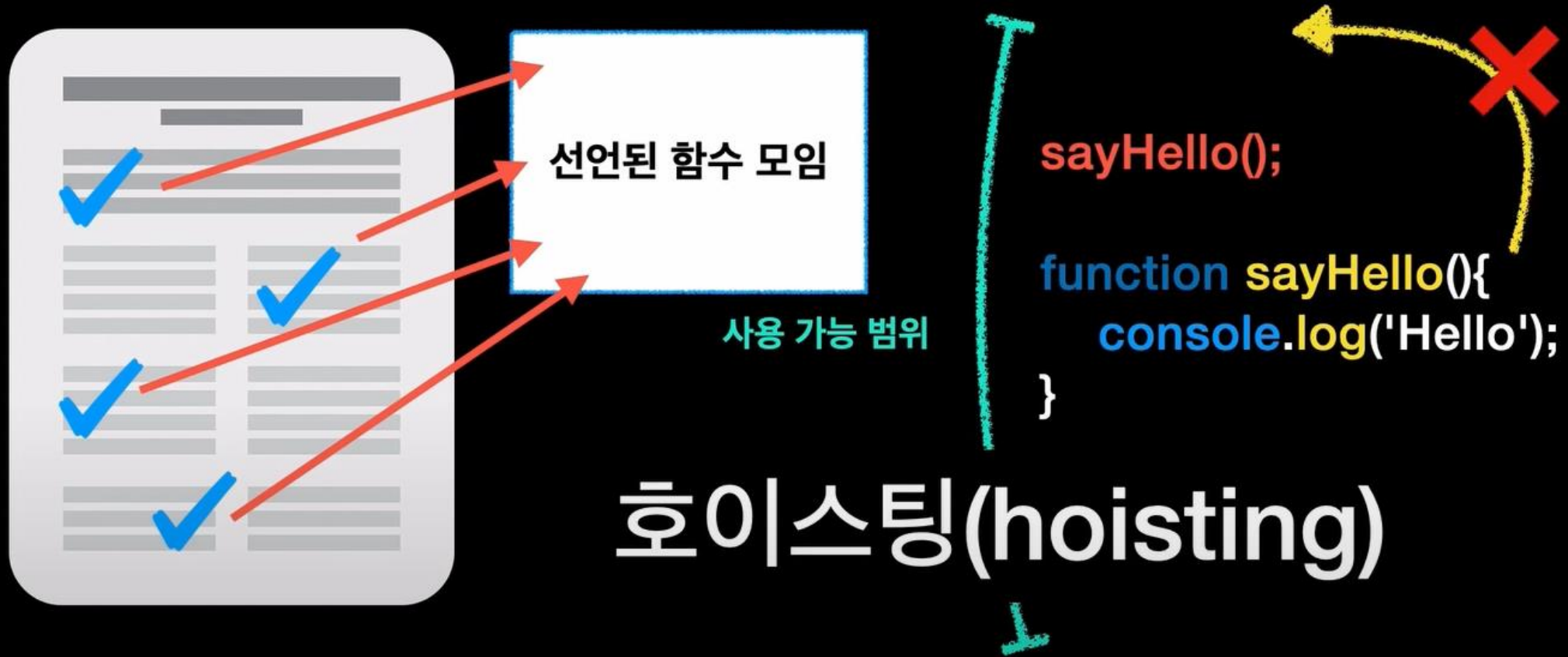
sayHello();



```
function sayHello(){  
    console.log('Hello');  
}
```



함수 선언문 : 어디서든 호출 가능





함수 표현식 : 코드에 도달하면 생성

1 ...

2 ...

3 `let sayHello = function(){` 생성
 `console.log('Hello');` 사용가능
 `}`

4 `sayHello();`



화살표 함수(arrow function)

```
let add = function(num1, num2){  
    return num1 + num2;  
}
```



화살표 함수(arrow function)

```
let add = (num1, num2) => {  
  return num1 + num2;  
}
```



화살표 함수(arrow function)

```
let showError = () => {  
    alert('error!');  
}
```

화살표 함수로 만들어 보기



```
// 함수 선언문
function sayHello(name) {
  console.log(`Hello, ${name}`);
}

// 함수 표현식
let sayHelloExpression = function (name) {
  console.log(`Hello, ${name}`);
};

// 화살표 함수
let sayHelloArrow = (name) => {
  console.log(`Hello, ${name}`);
};
```

Hello, Tetz

실습, 함수 선언 방법 연습하기



- 다음의 함수들을 각각 함수 표현식과 화살표 함수로 변경해 주세요

```
function add(a, b) {  
  return a + b;  
}  
  
function toUpperCase(str) {  
  return str.toUpperCase();  
}  
  
function sumArray(arr) {  
  let sum = 0;  
  for (let i = 0; i < arr.length; i++) {  
    sum += arr[i];  
  }  
  return sum;  
}
```



OnClick 으로

요소에 함수 걸어주기

onclick



- 각각의 HTML 요소에 속성 값으로 JS 함수를 연결

```
<body>
  <div class="box" onclick="test();">click</div>
</body>
```

```
function test() {
  alert("TEST");
}
```

이 페이지 내용:

TEST

확인



배열

Array



배열(array)
: 순서가 있는 리스트

Array



index

0

1

29

1번에 철수
2번에 영희

...

30번에 영수

```
let students = ['철수', '영희', ... '영수'];
```

```
console.log(students[0]); // 철수
```

```
console.log(students[1]); // 영희
```

```
console.log(students[29]); // 영수
```

```
students[0] = '민정';
```

```
console.log(students) // ['민정', '영희', ...
```

배열은 문자 뿐만 아니라, 숫자, 객체, 함수 등도 포함할 수 있음



```
let arr = [  
  '민수',  
  3,  
  false,  
  {  
    name : 'Mike',  
    age : 30,  
  },  
  function(){  
    console.log('TEST');  
  }  
];
```



배열에서 제공하는 기능



length : 배열의 길이

students.length // 30



push() : 배열 끝에 추가

```
let days = ['월', '화', '수'];
```

```
days.push('목')
```

```
console.log(days) // ['월', '화', '수', '목']
```



pop() : 배열 끝 요소 제거

```
let days = ['월', '화', '수'];
```

```
days.pop()
```

```
console.log(days) // ['월', '화']
```



shift, unshift 배열 앞에 제거/추가

추가

```
days.unshift('일');  
console.log(days) // ['일', '월', '화', '수'];
```

제거

```
days.shift();  
console.log(days) // ['월', '화', '수'];
```



배열의 메서드들 +

메소드 체이닝



```
let helloTest = "H-e-l-l-o";
let helloArr = helloTest.split("-");
// .split: 문자를 인수 기준으로 쪼개서 배열로 반환
// .reverse: 배열의 순서를 뒤집어서 반환
// .join: 배열을 인수 기준으로 문자로 병합해서 반환

console.log(helloArr);

let hello = "Hello~"
helloArr = hello.split("");
console.log(helloArr);

let reverseHello = helloArr.reverse();
console.log(reverseHello);

let helloAgain = reverseHello.join("");
console.log(helloAgain);

console.log(hello.split("").reverse().join(""));
```

```
▶ (5) ['H', 'e', 'l', 'l', 'o']
▶ (6) ['H', 'e', 'l', 'l', 'o', '~']
▶ (6) ['~', 'o', 'l', 'l', 'e', 'H']
~olleH
~olleH
```

실습, 배열 함수 연습하기



```
const exArr = ['T', 'T', 'e', 't'];
```

- 위의 배열에 대해서 아래의 작업을 완성하는 코드를 작성 하세요
- 배열 제일 앞의 값 제거하기
- 배열 제일 뒤에 'z' 문자열 추가하기
- 해당 배열을 하나의 문자열로 합치기
- 해당 문자열을 출력하기



객체!

Object



Object



Superman

name : clark
age : 33

키(key)

```
const superman = {  
  name : 'clark',  
  age : 33,  
}
```

값(value)



Object - 접근, 추가, 삭제

접근

superman.name // 'clark'

superman['age'] // 33

```
const superman = {  
  name : 'clark',  
  age : 33,  
}
```

추가

superman.gender = 'male';

superman['hairColor'] = 'black';

삭제

delete superman.hairColor;



// 객체 생성

```
let superman = {  
  name: "Clark",  
  age: 33  
}
```

// 객체 접근

```
console.log(superman.name);  
console.log(superman.age);
```

// 객체 데이터 추가

```
superman.gender = "M";  
superman["height"] = 187;  
console.log(superman);
```

// 객체 데이터 삭제

```
delete superman.height;  
console.log(superman);
```

Clark

33

▶ {name: 'Clark', age: 33, gender: 'M', height: 187}

▶ {name: 'Clark', age: 33, gender: 'M'}



Object - 프로퍼티 존재 여부 확인

```
const superman = {  
  name : 'clark',  
  age : 33,  
}
```

```
superman.birthDay;  
// undefined
```

```
'birthDay' in superman;  
// false
```

```
'age' in superman;  
// true
```



```
// 객체 생성
let superman = {
  name: "Clark",
  age: 33
}

console.log(superman.birthDay);
console.log('name' in superman);
console.log('height' in superman);
```

```
undefined
```

```
true
```

```
false
```



for ... in 반복문

```
for(let key in superman){  
  console.log(key)  
  console.log(superman[key])  
}
```



```
let superman = {  
  name: "Clark",  
  age: 33,  
  height: 187,  
  weight: 77,  
}  
  
for(key in superman) {  
  console.log(key);  
  console.log(superman[key]);  
}
```

name
Clark
age
33
height
187
weight
77



객체 Object

메소드 Method



Object



Superman

name : clark
age : 33

```
const superman = {  
  name : 'clark',  
  age : 33,  
  fly : function( ){  
    console.log('날아갑니다.')  
  }  
}
```



Object

method : 객체 프로퍼티로 할당 된 함수

superman.fly():
// 날아갑니다.

```
const superman = {  
  name : 'clark',  
  age : 33,  
  fly : function( ){  
    console.log('날아갑니다.')  
  }  
}
```



Object

```
const superman = {  
  name : 'clark',  
  age : 33,  
  fly : function( ){  
    console.log('날아갑니다.')  
  }  
}
```



Object

```
const superman = {  
  name : 'clark',  
  age : 33,  
  fly( ){  
    console.log('날아갑니다.')  
  }  
}
```



this



Object

```
const user = {  
  name : 'Mike',  
  sayHello : function(){  
    console.log(`Hello, I'm ${user.name}`);  
  }  
}
```

Object



```
const user = {  
  name : 'Mike',  
  sayHello : function() {  
    console.log(`Hello, I'm ${this.name}`);  
  }  
}
```

```
user.sayHello();
```

```
// Hello, I'm Mike
```



```
let boy = {  
  name: "Mike",  
  sayHello  
}  
  
let girl = {  
  name: "Jane",  
  sayHello  
}  
  
function sayHello() {  
  console.log(`Hello, I'm ${this.name}`);  
}  
  
boy.sayHello();  
girl.sayHello();
```

```
Hello, I'm Mike
```

```
Hello, I'm Jane
```




화살표 함수와

this



```
let sayHello = () => {  
  console.log(`Hello, I'm ${this.name}`);  
  console.log(this);  
}
```

```
let boy = {  
  name: "Mike",  
  sayHello  
}
```

```
let girl = {  
  name: "Jane",  
  sayHello  
}
```

```
boy.sayHello();  
girl.sayHello();
```

```
Hello, I'm dom.js:242
```

```
dom.js:243  
▶ Window {window: Window, self: Window, document: document, name: '', location: Location, ...}
```

```
Hello, I'm dom.js:242
```

```
dom.js:243  
▶ Window {window: Window, self: Window, document: document, name: '', location: Location, ...}
```



Object

화살표 함수는 일반 함수와는 달리 자신만의 **this**를 가지지 않음

화살표 함수 내부에서 **this**를 사용하면, **그 this는 외부에서 값을 가져 옴**

Object



```
let boy = {  
  name : 'Mike',  
  sayHello : () => {  
    console.log(this); // 전역객체  
  }  
}
```

- 브라우저 환경 : **window**
- Node js : **global**

```
boy.sayHello( );  
this !== boy
```





구조 분해 할당

(비구조화 할당)

구조 분해 할당



- 객체 또는 배열의 각각의 값을 분해하여 변수에 넣어 사용하는 방법

```
const user = {  
  id: 1,  
  name: "tetz",  
  email: "xenosign@naver.com",  
};  
  
const { id, name, email } = user;  
  
console.log(id);  
console.log(name);  
console.log(email);  
  
const fruits = ["사과", "딸기", "망고", "수박"];  
const [a, b, c, d] = fruits;  
console.log(a, b, c, d);
```



전개 연산자

전개 연산자



- 배열의 값을 , 단위로 구분 하여 전개 시켜주는 연산자

```
const fruits = ["사과", "바나나", "수박"];
console.log(fruits);
console.log(...fruits);
// console.log("사과", "바나나", "수박");

function conLog(a, b, c) {
  console.log(a, b, c);
}

conLog(fruits[0], fruits[1], fruits[2]);
conLog(...fruits);
```

나머지 연산자(매개 변수일 때)



- 매개 변수에 너무 많은 값이 들어올 때 나머지 연산자를 사용하여 한꺼번에 처리 가능

```
const fruits = ["사과", "바나나", "수박", "망고",  
"딸기"];
```

```
function conLog(a, b, ...c) {  
  console.log(a, b, c);  
}
```

```
conLog(...fruits);
```

```
const fruits = ["사과", "바나나", "수박", "망고", "딸기"];
```

```
function conLog(...rest) {  
  rest.forEach((element) => {  
    console.log(element);  
  });  
}
```

```
conLog(...fruits);
```

문자열을 배열로!



```
let string = "apple";  
let strToArr = [...string];  
console.log(strToArr);  
  
let strToArr2 = str.split("");  
console.log(strToArr2);
```

- 전개 연산자(...) 사용으로 "apple" 을 "a", "p", "p", "l", "e" 로 변환
- 그것을 [] 안에 넣어서 배열로 변환!

객체 병합!



```
const person = { name: '홍길동', age: 30 };  
const job = { title: '개발자', company: 'ABC Inc.' };  
  
const profile = { ...person, ...job, location: '서울' };  
console.log(profile);
```

- 전개 연산자는 객체에도 사용이 가능합니다!
- 객체 2개를 전개 연산자를 사용하여 하나의 객체로 합칠 수 있습니다!

실습, 다음 장의 코드를 주석에 맞게 완성하기



- 다음 장의 코드와 주석을 보고 내용에 맞게 코드를 완성해 주세요
- Console.log 의 결과는 다음과 같이 나오면 됩니다



```
const personalInfo = {  
  name: '이효석',  
  age: `Don't ask this :)`,  
  email: 'xenosign@naver.com',  
};
```

```
const jobInfo = {  
  position: '코딩 강사',  
  experience: '?년',  
};
```

```
// profile 이라는 하나의 객체로 저의 정보를 합쳐 주세요  
// profile 객체에 addr : '서대문구' 키와 값을 추가해 주세요
```

```
// 구조 분해 할당으로 각각 변수에 할당 해주세요
```

```
// 각 변수 콘솔에 출력  
console.log('이름:', name);  
console.log('나이:', age);  
console.log('이메일:', email);  
console.log('직책:', position);  
console.log('경력:', experience);  
console.log('지역:', addr);
```



원시 타입과 비원시 타입의 차이

객체의 불변성! (원시 타입과 비원시 타입)



- 불변성?
- 객체, 배열, 함수는 비원시 타입
- 데이터가 같은가? → X
- 메모리 주소가 같은가? → 0

```
const tetz = {  
  name: "이효석",  
  email: "xenosign@naver.com",  
};  
  
const 이효석 = {  
  name: "이효석",  
  email: "xenosign@naver.com",  
};  
  
console.log(tetz === 이효석); // FALSE
```


원시 (Primitive) 타입

String

Number

Boolean

Null

Undefined

BigInt

Symbol

객체 (Object) 타입

원시 타입을 제외한 모든 것

Object

Array

...

원시 (Primitive) 타입

```
const location = "korea"
```

location

"korea"

객체 (Object) 타입

```
const location = {  
  country: "korea"  
}
```

location

#12345



#12345

{ country: "korea" }

원시 (Primitive) 타입

```
const locationOne = "korea"
```

```
const locationTwo = "korea"
```

```
locationOne === locationTwo
```

```
> true
```

객체 (Object) 타입

```
const locationOne = {  
  country: "korea"  
}
```

```
const locationTwo = {  
  country: "korea"  
}
```

```
locationOne === locationTwo
```

```
> false
```

객체의 불변성! (원시 타입과 비원시 타입)



- 객체를 복사하면?!
- 객체의 시작 메모리가 전달
- 따라서, 동일한 객체가 됩니다.
- 복사한 객체를 변경하면?
- 원래 객체도 변경!

```
const tetz = {  
  name: "이효석",  
  email: "xenosign@naver.com",  
};  
  
const tetzCopy = tetz;  
tetzCopy.name = "tetz";  
  
console.log(tetzCopy);  
console.log(tetz);  
console.log(tetz === tetzCopy); // TRUE
```

